Laboratório Prolog · Aula 13

Thiago Fernando Zulli · RA 24000965-2 · Turma B
Gerado em 20/10/2026, 20:18:50 · tempo registrado no cronômetro: 38:31

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) → L = scheme. paradigma(java, logico) → false.

Saída do ola.pl

```
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
?- paradigma(java,A).
   paradigma(java,orientado_a_objetos)
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) pergunta qual é o paradigma do prolog (P é variável e recebe o valor logico). paradigma(prolog, logico) pergunta se isso é verdade (true ou false).

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Quer dizer que o programa não sabe isso (não consegue provar). Na base, Java é orientado_a_objetos, e não há fato dizendo que é lógico; false não é um fato sobre Java.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem): 7 respostas (lucia, marta, tereza, joana, bia, ana, dora). (mae(M,F), mulher(F)): 4 pares: lucia-tereza, marta-joana, tereza-bia, joana-ana.

Saída do familia.pl

```
?- mae(joana,jake).   → mae(joana,jake)
?- mae(jake,joana).   → false.
?- pai(maria,jake).   → false.
?- mae(joana,A).   → mae(joana,jake) / mae(joana,ana)
?- pai(A,jake).   → pai(jorge,jake)
?- mulher(A).   → lucia, marta, tereza, joana, bia, ana, dora
?- mae(joana,A),homem(A).   → mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).   → pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
```

```
   mae(joana,ana),mulher(ana)
?- mae(A,B),homem(B).
   mae(lucia,vern) / mae(marta,paulo) / mae(joana,jake) / mae(dora,leo)
?- homem(A),mulher(A).   → false.
?- mae(joana,jake),mae(joana,ana).   → true (as duas valem)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

"Quem é mãe de alguém que é mulher?" Ou seja, pares mãe-filha. Teve 4 respostas.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

A hipótese do mundo fechado: o que não pode ser provado pela base é falso. Em situações reais isso é problema porque falta de informação vira "falso" (um dado não cadastrado é tratado como inexistente).

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

lucia, marta, tereza, joana, bia, ana, dora. A ordem é a dos fatos na base, de cima para baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake): X = paulo. primo(leo, X): X = jake e X = ana.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Falham: f(X,X)=f(a,b), [A|B]=[] e 2+3=5. Unifica: f(X,X)=f(a,Y) (X=a, Y=a).

Saídas de regras.pl e unificacao.pl

```
--- regras.pl
?- avo(A,jake).   → avo(marta,jake), avo(vern,jake)
?- avo(osvaldo,A).   → joana, paulo, bia
?- irmao(jake,A).   → irmao(jake,ana)
?- tio(A,jake).   → tio(paulo,jake)
?- primo(leo,A).   → jake, ana
?- ancestral(osvaldo,A).   → vern, tereza, joana, paulo, jake, ana, leo, bia
?- ancestral(A,jake).   → joana, jorge, lucia, marta, osvaldo, vern
?- ancestral(jake,A).   → false.
?- irmao(A,B).   → vern-tereza, tereza-vern, joana-paulo, paulo-joana, jake-ana, ana-jake
?- tio(A,B).   → tio(vern,bia), tio(paulo,jake), tio(paulo,ana)
?- ancestral(A,leo).   → dora, paulo, lucia, marta, osvaldo, vern
?- primo(A,B).   → bia-joana, bia-paulo, jake-leo, ana-leo, joana-bia, paulo-bia, leo-jake,
leo-ana
[vern,tereza,joana,paulo,jake,ana,leo,bia]

--- unificacao.pl
a = a  unifica
a = b  NAO unifica
A = a  unifica   A = a
f(A,b) = f(a,B)  unifica   A = a   B = b
f(A,A) = f(a,b)  NAO unifica
f(A,A) = f(a,B)  unifica   A = a   B = a
pai(A,jake) = pai(jorge,B)  unifica   A = jorge   B = jake
[A|B] = [1,2,3]  unifica   A = 1   B = [2,3]
[A|B] = []  NAO unifica
[A,B|C] = [a,b,c,d]  unifica   A = a   B = b   C = [c,d]
```

```
2+3 = 5  NAO unifica
f(A,B) = f(1,2)  unifica   A = 1   B = 2
g(A,b) = g(a,A)  NAO unifica
[A,B|C] = [1,2,3,4,5]  unifica   A = 1   B = 2   C = [3,4,5]
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Uma cláusula só não alcança avós, bisavós etc. Caso base: ancestral(X,Y) :- progenitor(X,Y). Caso recursivo: ancestral(X,Y) :- progenitor(X,Z), ancestral(Z,Y).

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Passou a aparecer irmao(jake,jake): jake virou irmão dele mesmo. X \== Y é necessária para impedir que alguém seja irmão de si próprio.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

No primeiro, X teria de ser a e b ao mesmo tempo, o que é impossível. No segundo, X = a e Y fica igual a X, então X = a e Y = a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Não. Uma função de Java ou Scheme tem entradas e saída fixas, só calcula numa direção. No Prolog avo é uma relação: qualquer argumento pode ser variável, então serve para achar avós ou netos.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

4 respostas: ([],[1,2,3]), ([1],[2,3]), ([1,2],[3]) e ([1,2,3],[]).

Saída do listas.pl

```
?- membro(b,[a,b,c]).   → true
?- membro(z,[a,b,c]).   → false.
?- membro(A,[a,b,c]).   → a, b, c
?- tamanho([a,b,c],A).   → 3
?- tamanho([],A).   → 0
?- concatena([1,2],[3,4],A).   → [1,2,3,4]
?- concatena(A,[3],[1,2,3]).   → A = [1,2]
?- concatena([1],A,[1,2,3]).   → A = [2,3]
?- concatena(A,B,[1,2,3]).
   [] e [1,2,3]
   [1] e [2,3]
   [1,2] e [3]
   [1,2,3] e []
?- inverte([1,2,3],A).   → [3,2,1]
?- concatena(A,[b,c],[a,b,c]).   → A = [a]
?- inverte([a,b,c,d],A).   → [d,c,b,a]
?- tamanho([x,y],A).   → 2
?- membro(A,[]).   → false.
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Caso base: tamanho([], 0). Passo: tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1. A cada chamada fica pendente o N is M + 1, que só é calculado na volta, depois de chegar na lista vazia (0, 1, 2, 3).

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Porque há 4 jeitos de dividir uma lista de 3 elementos em prefixo e sufixo (com 0, 1, 2 ou 3 elementos no início). O anexar do Scheme só junta duas listas (uma direção); não consegue separar uma lista.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O backtracking: o Prolog tenta a 1ª cláusula (X é a cabeça: a), depois volta e tenta a 2ª (procura na cauda), repetindo até esgotar a lista (b, c).

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3 deixa X com a estrutura 2+3 (não calcula). X is 2 + 3 dá X = 5. fatorial(N, 120) dá erro de instanciação.

Saída do aritmetica.pl

```
?- A=2+3.   → 2+3
?- A is 2+3.   → 5
?- 5=:=2+3.   → true
?- 2+3=5.   → false.
?- A is 7 mod 3.   → 1
?- A is 7//2.   → 3
?- A is 2^10.   → 1024
?- fatorial(5,A).   → 120
?- fatorial(0,A).   → 1
?- fib(10,A).   → 55
?- soma([1,2,3,4],A).   → 10
?- fatorial(A,120).   → ERRO: instantiation_error
?- A is B+1.   → ERRO: instantiation_error
?- A is 5*4+2.   → 22
?- A=5*4+2.   → 5*4+2
?- fatorial(10,A).   → 3628800
?- fib(20,A).   → 6765
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= unifica sem calcular: X = 2+3 deixa X como a estrutura 2+3. is calcula o lado direito e unifica: X is 2+3 dá X = 5. =:= compara valores numéricos: 5 =:= 2+3 é true.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

Em fatorial, N > 0 e N1 is N - 1 precisam de N conhecido; com N livre dá erro de instanciação. concatena só usa unificação, que funciona em qualquer direção. A aritmética (is, >) não é relacional.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Igual: a mesma estrutura recursiva (caso base fatorial 0 = 1 e n * fatorial(n-1)). Diferente: em Haskell o resultado é o valor devolvido; em Prolog é um argumento extra (F), precisa de is para calcular e só funciona com N conhecido.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) dá false; solteiro1(X) dá respostas. maximo2(5,3,M): duas respostas (5 e 3). maximo3(5,3,3): true (errado).

Saída de controle.pl e mensagem de ancestral_ruim

```
--- controle.pl
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(ana,caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio,ana)
   tenta caio e bia
   tenta caio e caio
?- solteiro1(A).   → osvaldo, paulo, jake
?- solteiro2(A).   → false.
?- maximo1(5,3,A).   → 5
?- maximo2(5,3,A).   → 5 e 3
?- maximo3(5,3,A).   → 5
?- maximo1(5,3,3).   → false.
?- maximo3(5,3,3).   → true (errado)
?- maximo3(8,3,A).   → 8
?- maximo2(8,3,A).   → 8 e 3

--- laco_infinito.pl
ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)
ERROR: Stack limit (1.0Gb) exceeded (estouro de pilha)
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

3 cores: 24 colorações. 2 cores: nenhuma (PR, SP e MS fazem fronteira entre si e precisam de 3 cores).

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
   false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

Com X = ana e Y = ana, a meta X \== Y falhou. O Prolog voltou ao último ponto de escolha (amigo(Y)) e tentou bia, depois caio, e quando acabaram voltou a amigo(X). Achou ana-caio e caio-ana (os dois gostam de café).

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

solteiro2 testa \+ casado(X) com X ainda sem valor: casado(X) é provável (vern), então a negação falha e dá false. solteiro1 primeiro gera X com homem(X) e depois testa. Osvaldo é "solteiro" porque casado(osvaldo) não está na base (mundo fechado), mesmo sendo pai.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

Na 1ª cláusula, maximo3(X, Y, X), o 3º argumento (3) forçaria X = 3, mas X = 5: a cabeça não unifica, então ela falha antes do corte e o corte nunca roda. Aí a 2ª cláusula, maximo3(_, Y, Y), aceita 3. Correção: maximo3(X, Y, Z) :- X >= Y, !, Z = X. e maximo3(_, Y, Y).

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

Em ancestral_ruim a primeira meta chama a própria regra com os mesmos argumentos, sem consumir nada (recursão à esquerda): laço infinito. Em ancestral_bom a recursão vem depois de progenitor, que avança na árvore. A lógica é a mesma, mas a ordem das metas muda a execução.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

3 cores: 24; 2 cores: nenhuma; 4 cores: 1728. É declarativo porque só descreve o que é uma coloração válida (restrições), sem laço nem algoritmo; a busca fica por conta do backtracking.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

Funcional: anexar é uma função com entradas e saída fixas, só junta listas. Lógico: concatena é uma relação, serve para juntar, separar e testar, em várias direções, com várias respostas por backtracking.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

1) Controle da ordem de resolução: laco_infinito.pl (ancestral_ruim estoura a pilha). 2) Negação/mundo fechado: controle.pl (solteiro1 acha osvaldo solteiro e solteiro2 dá false).

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas de regras, busca e restrições (sistemas especialistas, bancos de dados dedutivos, linguagem natural, mapas/horários). Não escolheria para cálculo numérico pesado, interfaces ou programas com muita entrada/saída, pois a aritmética não é relacional e a busca pode ser lenta ou infinita.